CoreSyS Deep Report

1. Overview

CoreSyS is a 64-bit operating system designed for UEFI environments, written primarily in a custom language called Technical C (TC). The OS focuses on providing a minimalistic, efficient, and modular system suitable for both experimental and practical applications. It emphasizes a direct approach to low-level system control while maintaining modern OS functionalities.

2. Architecture

2.1 Kernel

- Located at `CoreSyS/src/CoreSys/kernel/main.tc`.
- Handles core system services, process management, and memory management.
- Provides an interface for UEFI boot services.

2.2 Boot Process

- Direct interaction with UEFI firmware using `ExitBootServices()`.
- Memory mapping is performed via `GetMemoryMap()` to understand available physical memory.
- Supports `.efi` and `.bin` executables, using NE headers for system binaries.

2.3 File System

- Utilizes UEFI-compliant file structures.
- Supports modular loading of `.mod` (DLL), `.tc` (Technical C source), `.th` (header), and `.run` (executables).

2.4 Memory Management

- Provides granular control of memory regions via UEFI memory maps.
- Kernel operates with direct memory access capabilities.

2.5 Extensibility

- Supports user-level applications and drivers loaded dynamically.
- CoreSys modules (`.mod`) can extend system functionalities.
- Designed for experimentation with OS-level concepts, including custom UEFI drivers.

3. Programming Language: Technical C (TC)

- A low-level C-inspired language designed for OS development.
- Allows inline C integration.
- Emphasizes predictability, deterministic memory handling, and direct hardware interfacing.

4. System Utilities

- CoreSys includes a minimal set of utilities to manage hardware and system services.
- Designed to run without a GUI but can be extended with user interface modules.

5. Key Features

- 64-bit UEFI-native OS.
- Fully modular kernel design.
- Dynamic module loading with `.mod` files.
- Full control over memory, CPU, and storage hardware.
- Supports experimental OS research and secure system experiments.

6. Use Cases

- OS-level research and learning.
- Secure computing environments.

- Hardware interfacing and experimentation.
- UEFI application testing and driver development.

7. Documentation & Resources

- Source: `CoreSyS/src/CoreSys/`
- Technical C specifications for kernel and modules.
- UEFI documentation for boot services and memory management.

8. Conclusion

CoreSyS represents a modern experimental OS focused on UEFI systems. Its modular kernel, coupled with the Technical C language, makes it an ideal platform for learning OS internals, driver development, and secure system experiments.

Report generated for internal documentation purposes.